UNIVERSIDAD DE GUADALAJARA
Red Universitaria e Institución Benemérita de Jalisco

C# Winform



Formulario con WinForms



Alumno**: Magaña López Miguel Ángel **- 218096781

***Director:** MA. INES CALDERON ZETTER*

Centro Universitario de Ciencias Exactas e Ingenierías

División de Tecnologías para la Integración Ciber-Humana

CUCEI

***Fecha:** 3 / Octubre / 2026*

ÍNDICE.

Objetivo:.....2
Diseño del formulario:.....2
Código:.....3
Ejecución y pruebas:.....4

Funcionamiento de la Aplicación

Objetivo:

Realizar una aplicación de WinForms la cual permita ingresar una contraseña y confirmarla mediante una expresión regular donde se verifique que contenga al menos una letra mayúscula, una letra minúscula, un número y un símbolo y posteriormente se verifique que ambas contraseñas coincidan.

Diseño del formulario:

La interfaz gráfica fue diseñada mediante el Windows Forms Designer, utilizando el Toolbox para incorporar y organizar controles como Label, TextBox, Button y Panel, usando Visual Studio y .NET 8 para la creación de la app de escritorio.

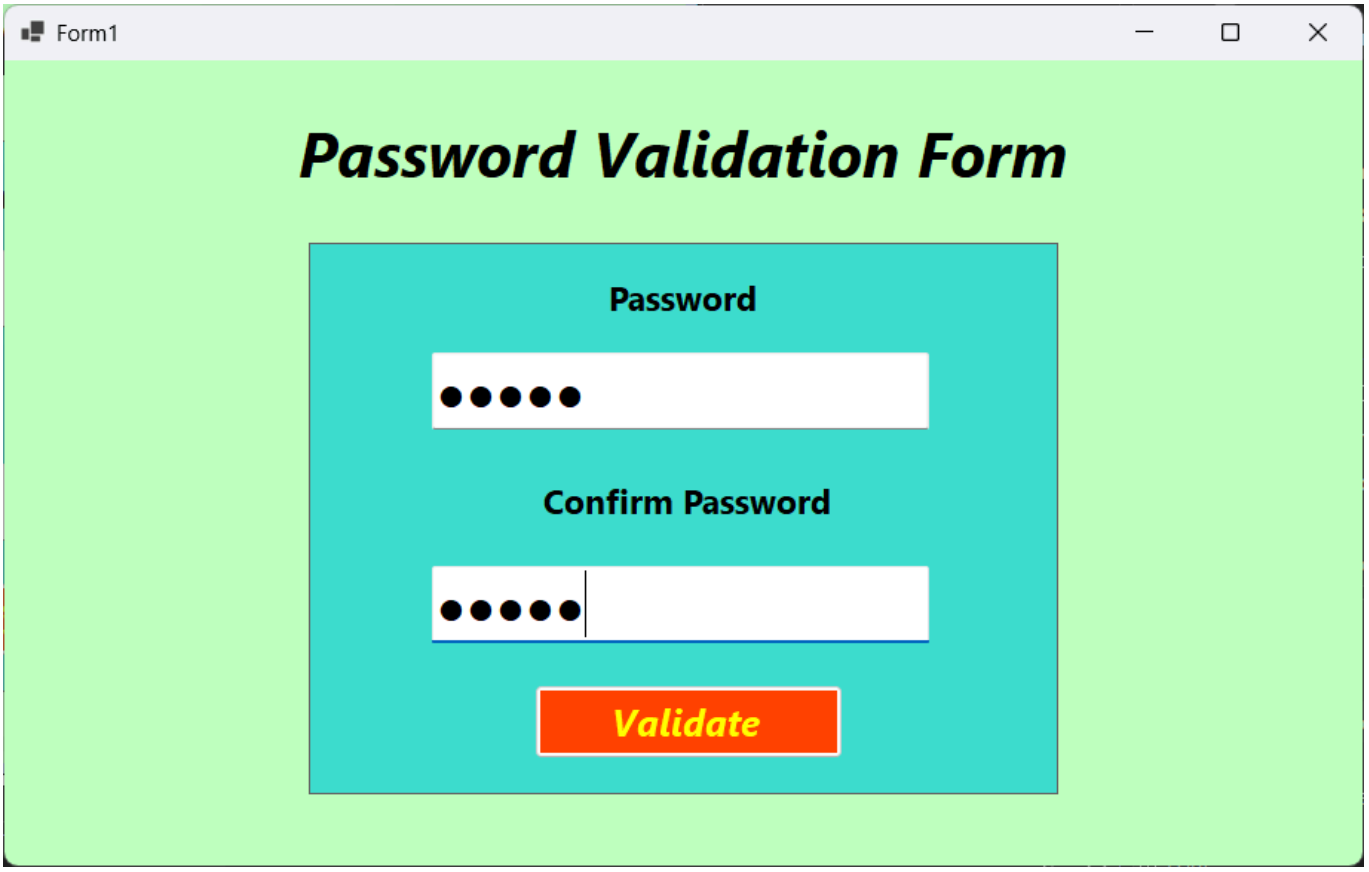


Imagen 1. Vista del diseño de la interfaz gráfica de usuario (GUI) del programa en Windows Forms.

Código:

Los controles se incorporaron al formulario utilizando el Toolbox mediante la técnica de arrastrar y soltar (drag and drop) donde al agregar o modificar estos controles, Visual Studio genera y actualiza automáticamente el código correspondiente en el archivo Form1.Designer.cs conteniendo su configuración visual.

Por otra parte, en el archivo **Form1.cs** se implementó la lógica de la aplicación, incluyendo el evento Click del botón, la expresión regular para validar la contraseña, la comparación de ambos campos y los mensajes mostrados al usuario, por lo que de esta manera, Windows Forms permite separar la construcción de la interfaz gráfica de la lógica de programación.

```
/ Expresión regular (Regex) la cual valida los requisitos de la contraseña
/ donde se tiene que:
/ (?=.*[A-Z])          Indica que se debe contener al menos una letra de la A-Z mayuscula
/ (?=.*[a-z])          Indica que se debe contener al menos una letra minúscula de la a-z
/ (?=.*\d)             Indica que se debe contener al menos un número del 0-9
/ (?=.*[^a-zA-Z\d])    Indica que se debe contener algo que no sea una letra en el rango a-z, A-Z o un numero,
/                      por lo que se validaria si hay un simbolo
/ .*                   Indica que se puede tener cualquier cantidad de cualquier caracter en la expresion


tring pattern = @"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[^a-zA-Z\d]).*$";
/ Las cuatro condiciones anteriores deben cumplirse al mismo tiempo
/ donde el orden de los requisitos es indistinto por los lookaheads (?=...)
/ al buscar en cualquier posición de la cadena
/ Finalmente, .* permite que la extension de la validacion se realize por toda la cadena
/ en cualquier cantidad de caracteres hasta el final
```

Imagen 2.1 Expresión regular con Regex para la validación de la contraseña

```
using System.Text.RegularExpressions;

namespace HolaMundo
{
    3 references
    public partial class Form1 : Form
    {
        1 reference
        public Form1()
        {
            InitializeComponent();
        }

        1 reference
        private void textBox1_TextChanged(object sender, EventArgs e)
        {

        }

        1 reference
        private void button1_Click(object sender, EventArgs e)
        {
            // Obtiene el texto del txtBox introducido wn la contraseña
            string password = txtPassword.Text;

            // Obtiene el texto del txtBox en la confirmación de contraseña
            string confirmPassword = txtConfirmPassword.Text;

            // Expresión regular (Regex) la cual valida los requisitos de la contraseña
```

Imagen 2.2 Código y declaración de funciones del archivo **Form1.cs**

Por lo que la función llamada **botton1_Click** es la encargada de manejar el evento onClick del botón mostrado en la interfaz gráfica la cual se realiza la validación de la contraseña al activarse y muestra un message box con la validación realizada.

Ejecución y pruebas:

Una vez finalizado el diseño de la interfaz y la implementación del código, se ejecutó la aplicación desde Visual Studio desde Iniciar (Start) para comprobar el funcionamiento del programa, en el cual durante la ejecución se realizaron

diferentes pruebas con distintas contraseñas para verificar que la expresión regular y las validaciones implementadas funcionaran correctamente.

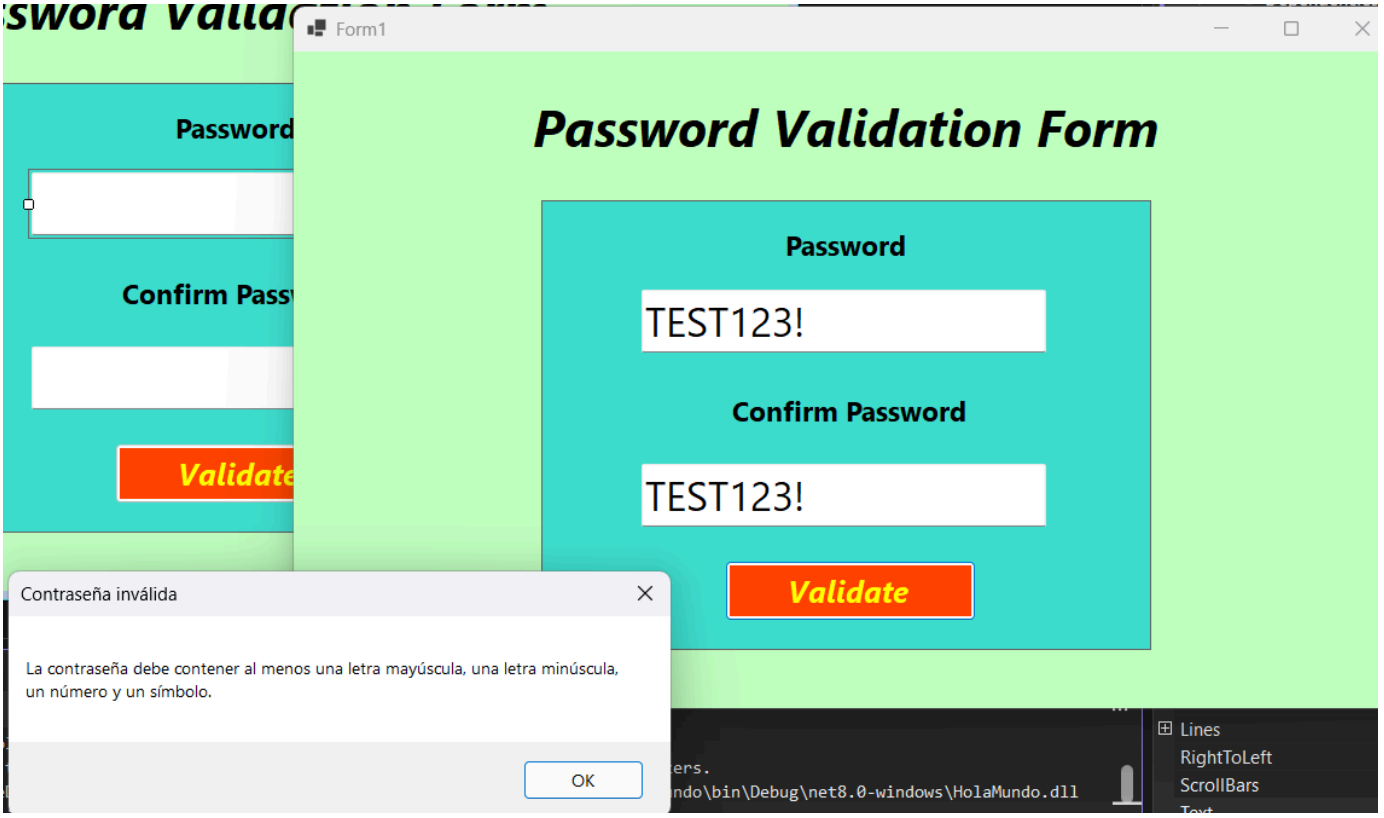


Imagen 3.1 Prueba de contraseñas que no cumplen con **minúsculas [a-z]** en la expresión regular.

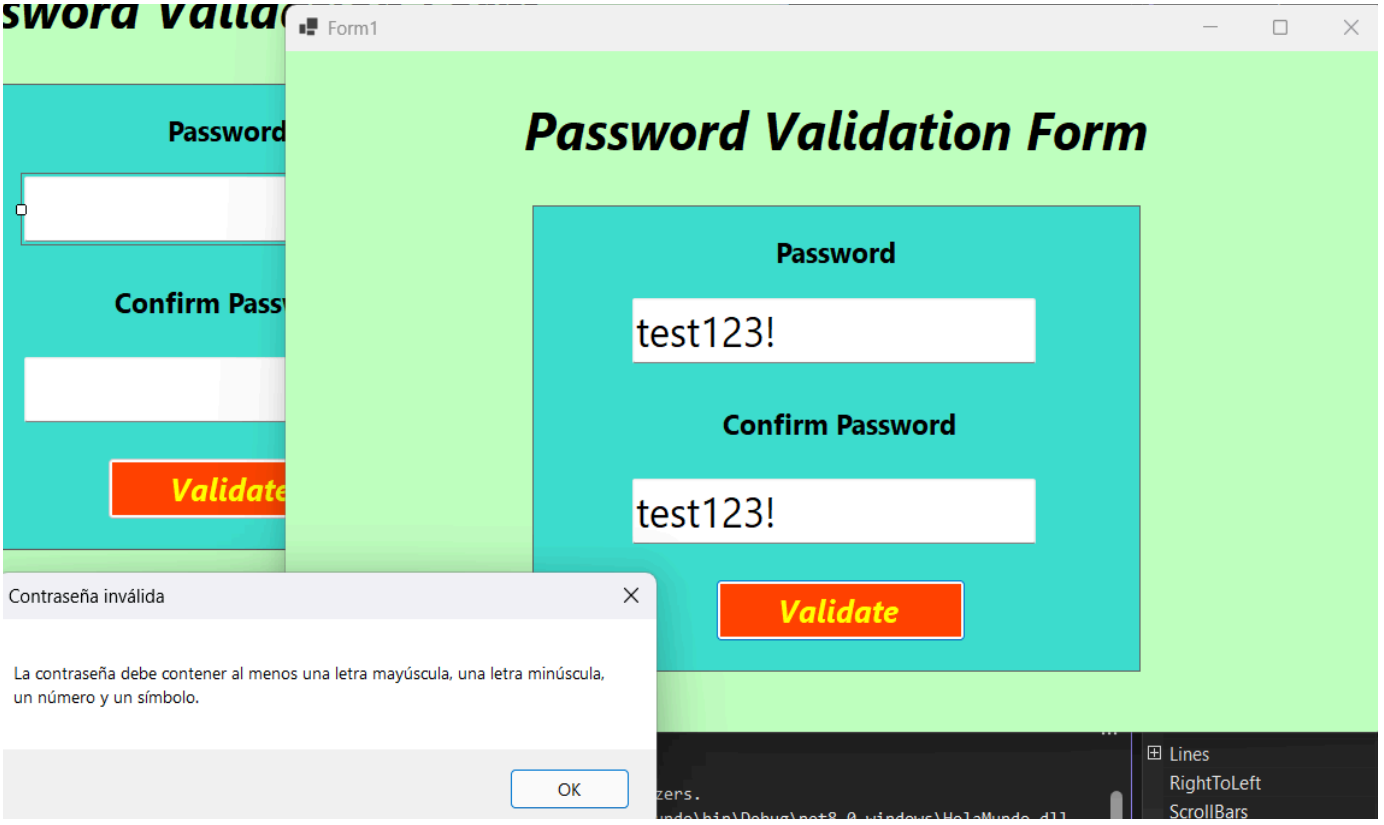


Imagen 3.2 Prueba de contraseñas que no cumplen con **mayúsculas [A-Z]** en la expresión regular.

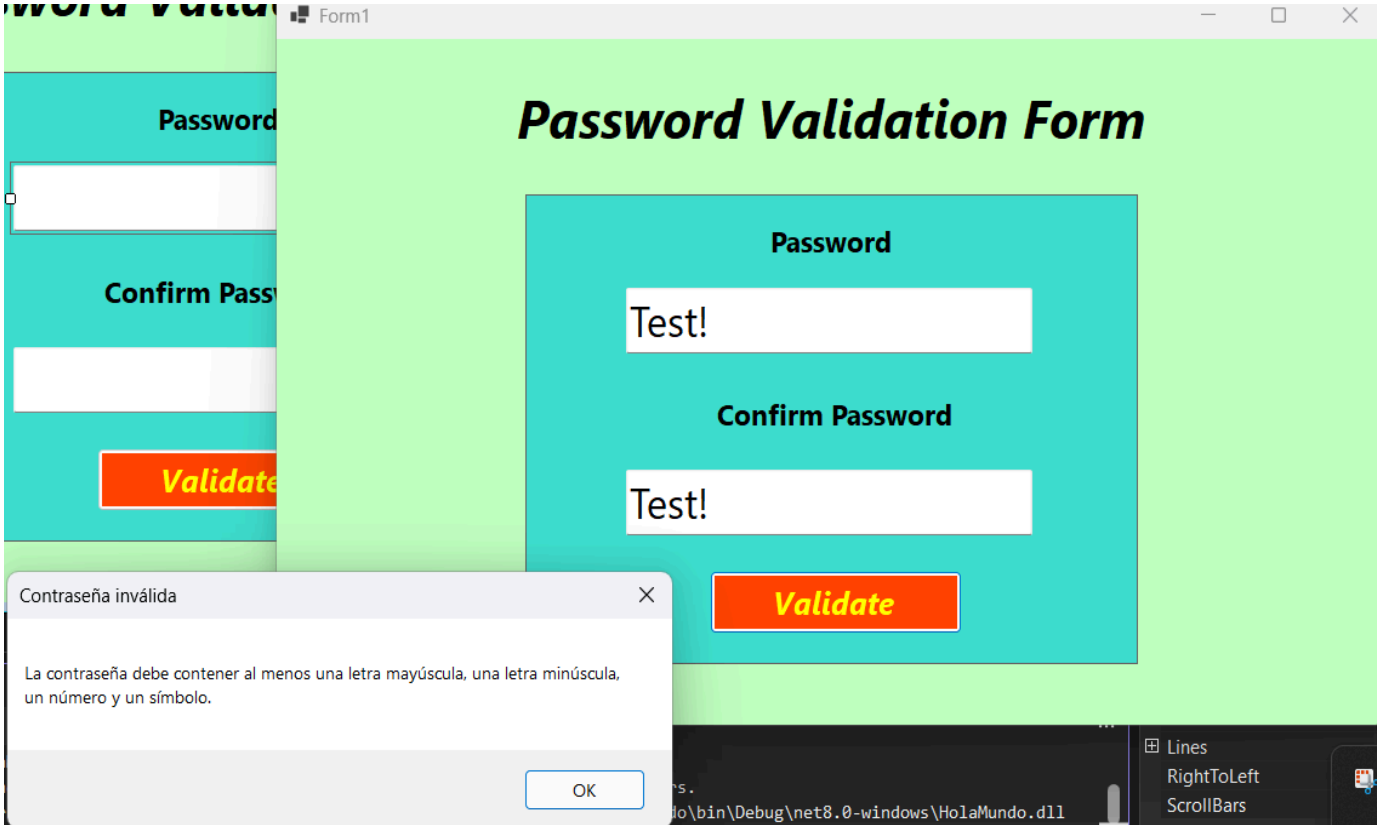


Imagen 3.3 Prueba de contraseñas que no cumplen con **dígitos [/d]** en la expresión regular.

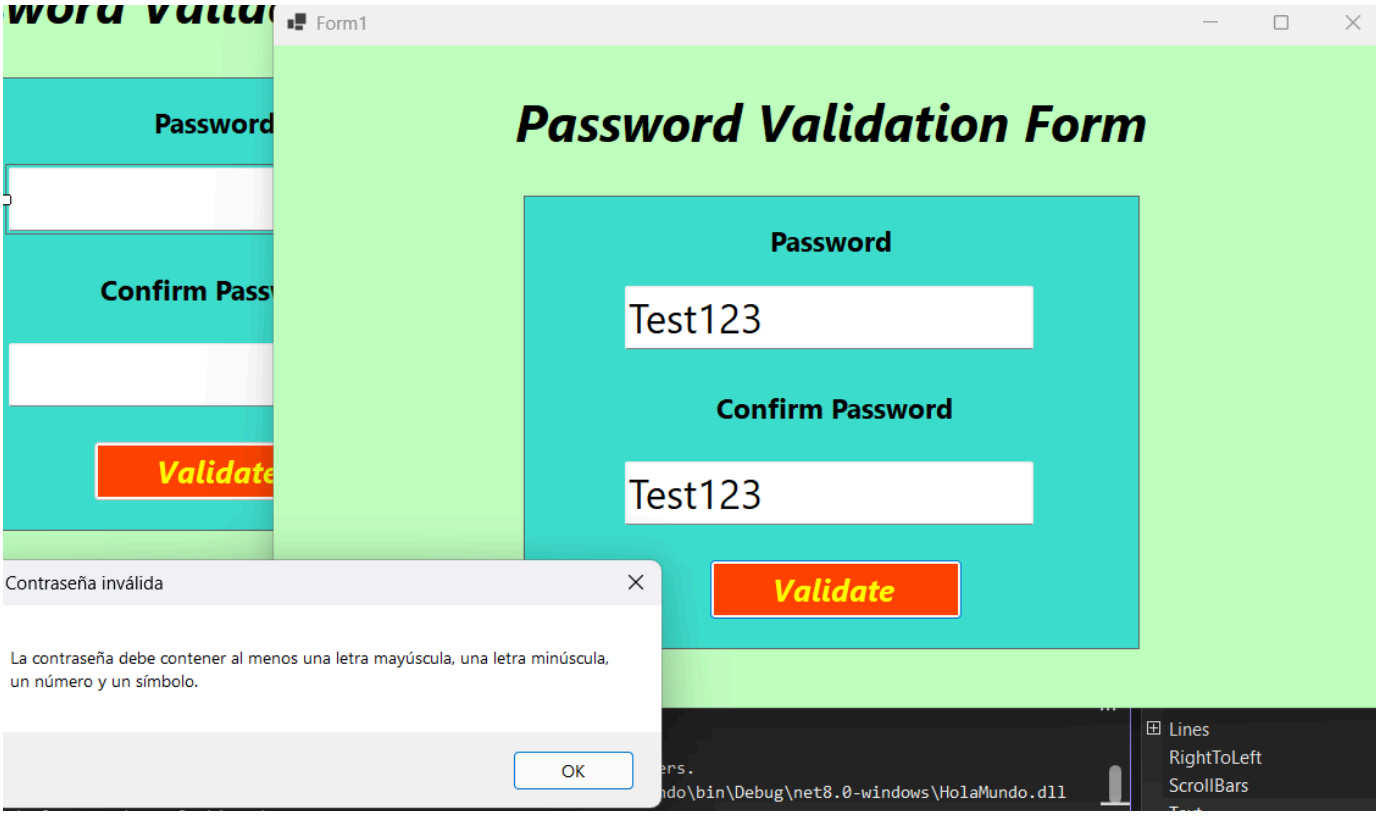


Imagen 3.4 Prueba de contraseñas que no cumplen con **símbolos [[^a-zA-Z\d]** en la expresión regular.

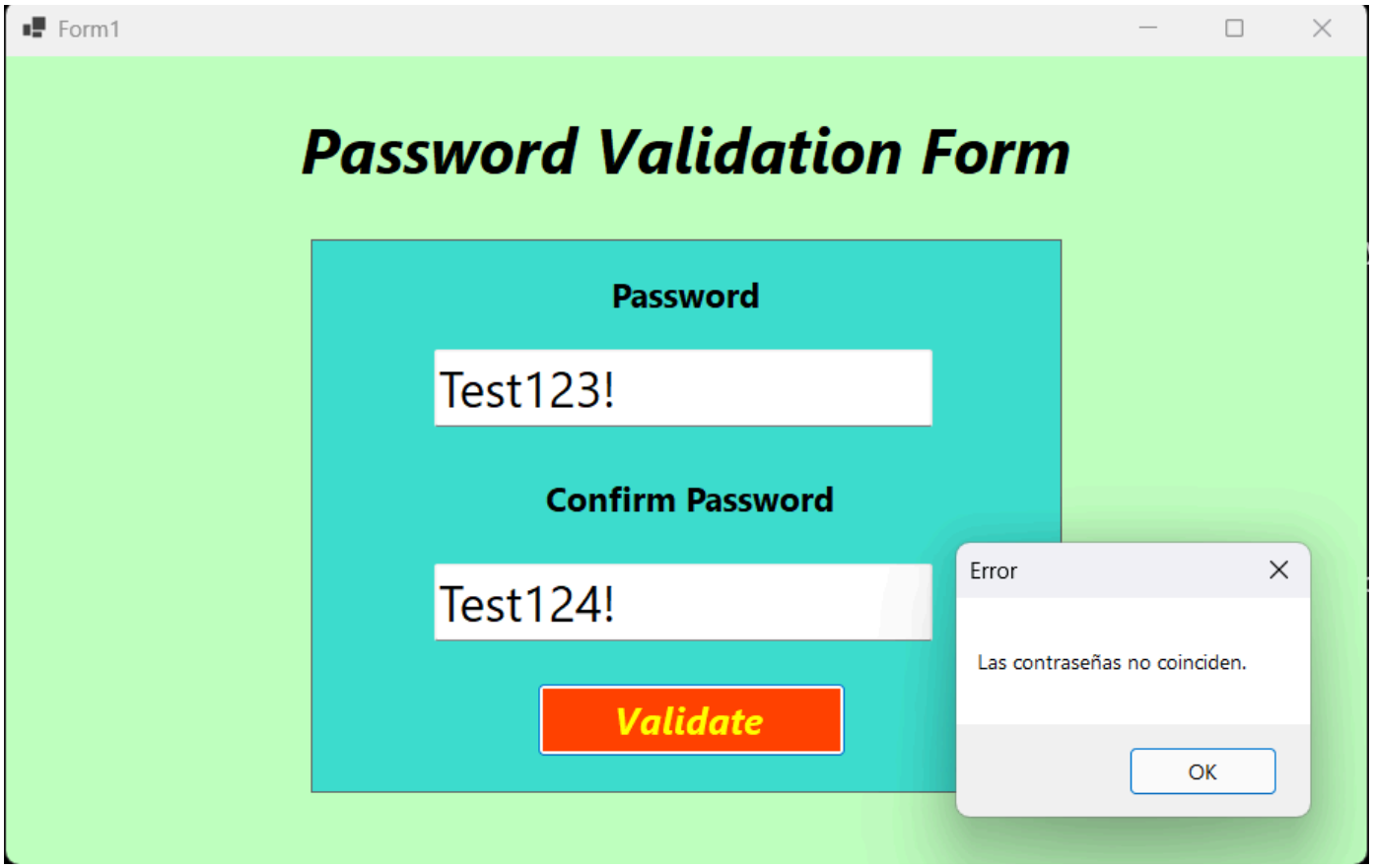


Imagen 3.5 Prueba de contraseñas que no coinciden

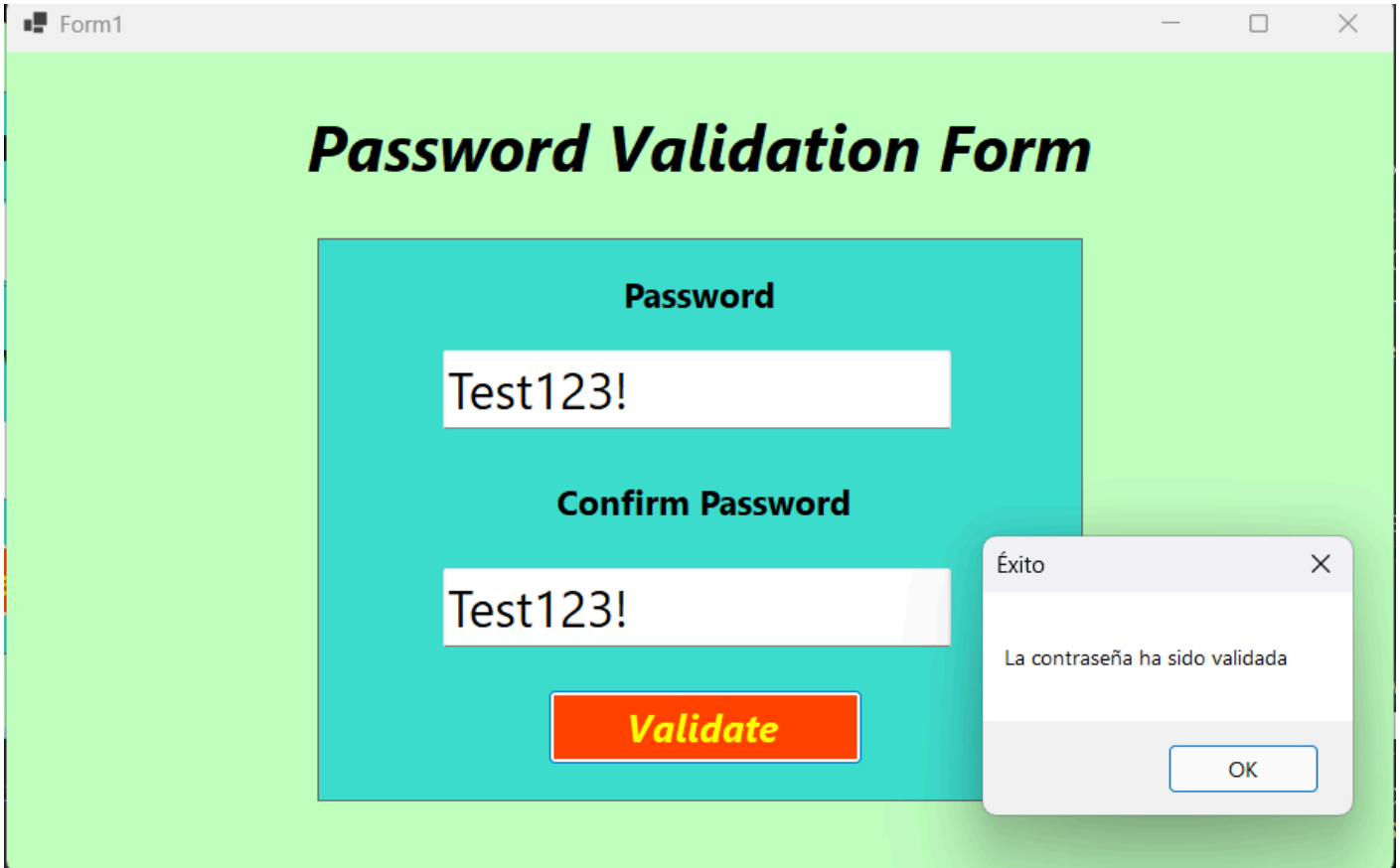


Imagen 3.6 Prueba de validación exitosa de contraseñas

Por lo que se puede verificar su correcto funcionamiento del programa en los tres distintos casos evaluados en la expresión regular implementada previamente:

```
string pattern = @"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[^a-zA-Z\d]).*$";
```

En ella, el orden de las validaciones representadas por los lookaheads (?=...) es indiferente, ya que cada uno verifica de manera independiente el cumplimiento de una de las cuatro condiciones requeridas, adicionalmente, **.*** (metacaracter . y la cerradura de Kleenn) permiten que el patrón buscado pueda encontrarse en cualquier posición de la cadena y no en una posición fija.